אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

August 2026

Abstract

A semantic cache answers a new query by returning the stored response to a sufficiently similar older one. Such caches are evaluated on hit rate, latency, and sometimes false-hit rate. None of those asks whether a served hit was still *true* when served. I name that quantity *stale-hit-rate*: the fraction of served hits whose content had already expired. FreCoS, the extension to GPTCache presented here, fits an exponential validity-decay rate per query cluster and then reads it in two places: as a hard validity gate on the serve path, and as a decay term in a cost-aware eviction value function. Cluster identity comes from k-means over embeddings from the cache's own encoder, so both consumers read only signals a deployed cache already has. The staleness table is the exception: it is fit against the generator's ground-truth expiry field, which a deployment would replace with a staleness detector or a stale-answer feedback channel.

FreCoS is not a faster cache. It is a freshness-aware cache that buys answer validity with hit rate and with spend. The main result is that the gate works on the metric it targets: on a seeded synthetic workload with per-cluster content half-lives, it cuts stale-hit-rate roughly eightfold against a frequency-based floor, with perfect separation across seeds. Second, fitting a rate per cluster beats pooling one rate over the whole cache, with a large and significant effect, while still falling short of an oracle that reads the true rates by a margin that is large but does not survive correction. Third, the cost-aware eviction term does not hold up: under real eviction pressure it is indistinguishable from a pure-frequency policy on cost saved, and plain LRU beats both. The gate pays for that with hit rate, which falls from 0.894 to 0.336, and with spend, which rises 6.45-fold. Counted per scored query rather than per served hit, useful hits move the other way and rise 61.3%. The latency and throughput regressions follow from serving fewer hits, each miss paying a simulated cost, rather than from the gate itself, whose measured overhead does not rise. One limitation bounds every absolute number here: false-hit-rate, made measurable by the semantic index, is 0.89–0.92 in every bracket-style experiment, so the claims are restricted to relative comparisons within an experiment.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response when an incoming query lands close enough in embedding space to one already answered. The promise: skip the call, return the same answer faster and for free. But "the same answer" is a claim about time as well as similarity, and most semantic caches never check the time part.

This matters most where the content behind an answer changes on its own schedule. Monitoring how a brand is represented across AI-generated answers is one such case: the same style of question is asked repeatedly, exactly the workload a semantic cache absorbs, but the facts behind it keep changing.

Two gaps follow, and FreCoS adds one component for each. The serve path has no notion of staleness: a cache that asks only "have I seen a similar question" and never "is that answer still true" serves a stale answer indefinitely once written, and "found in cache" is silently treated as "correct," because stale-hit-rate does not exist in standard cache evaluation. Eviction has no cost term: count-based policies such as LRU and LFU treat two entries with identical access patterns alike even when one is far more expensive to regenerate.

Existing systems address these gaps unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock TTL and no similarity matching, and semantic-caching products use one fixed global threshold with plain TTL or count-based eviction. The research literature is further ahead, proposing cost-aware and staleness-aware eviction formulas, but none names or measures stale-hit-rate. I compare against that literature, not the weaker product baseline.

This report makes three contributions. The first is stale-hit-rate as a named, measured metric. I measure it on a harness that uses a real cosine-similarity index and real k-means clustering instead of exact-match lookup and ground-truth cluster labels, so the cluster label the gate and the eviction term read is one a deployed cache could recover, as is every other signal on the serving path. The staleness table is the exception: in all three modes it is calibrated against a generator field a deployed cache cannot observe. The second contribution is evidence that per-cluster staleness fitting does measurable work under controlled conditions: learned rates beat a pooled rate with a large, significant effect, and fall a large margin short of an oracle that reads the generator's true rates, which is what a fitter working off an imperfect but real signal should do. The third is a negative result on cost-aware eviction, measured under real eviction pressure: FreCoS's cost term is indistinguishable from a pure-frequency floor, and plain LRU beats both, for a reason Section 5.3 traces to the workload rather than to any staleness mechanism.

All of that has a price. FreCoS is not a faster cache: it buys answer validity with hit rate and with spend. Gating cuts hit rate from 0.894 to 0.336 and pays 6.45x the spend for a 1.52x rise in cost saved (Table 5). The latency and throughput regressions in that table are a consequence of serving fewer hits, each miss paying a simulated cost, and not of the gate, whose measured overhead per decision does not rise (Section 5.2).

One thing limits how far any of those numbers travel. False-hit-rate on this index is 0.89–0.92, so the absolute usefulness of the cache on this workload is poor, and every claim here is restricted to relative comparisons within an experiment. The gate barely moves that rate (0.922 with the gate off against 0.911 with it on), so it belongs to the index and the workload, not to anything FreCoS adds, and the similarity threshold most likely to move it is never swept (Table 1).

1.2 Related work

The closest prior art, and my primary eviction comparator, is Biton and Friedman's work on semantic-cache eviction [1]. It proves optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality, evaluated on GPTCache with released code. Two of their findings shape my design: LRU is weak on semantic workloads, and frequency-based policies are a stronger baseline, which is why every comparison here treats an LFU-class policy, not LRU, as the floor. FreCoS adds two orthogonal signals, regeneration cost and content validity over time, and evaluates against a correctness metric their setting never defines.

The structurally closest published formula is Cortex's LCFU policy [2], scoring an entry as log-scaled frequency times cost times latency times a staleness term, divided by size, with a TTL purge on top. The difference is narrow but it matters: Cortex's staleness term is a static score a language model assigns at write time, while FreCoS learns a decay rate per cluster from observed staleness, and Section 5.1 tests whether that learning does measurable work. I did not reimplement LCFU, since it relies on remote tool-call latency metadata with no equivalent here.

Several other lines touch individual pieces without matching the combination: GDSF [3] ages access recency, not content validity; Category-Aware caching [4] assigns load-based, not learned, per-category TTLs at the same granularity as my per-cluster ones; FreshCache [5] puts staleness in the serve path as a probabilistic risk budget rather than a hard gate; SCALM [6] clusters entries as my k-means step does but with no staleness-aware significance score; MeanCache [7] is cost-motivated but does not target eviction; W-TinyLFU [8] is admission control, orthogonal to both my contributions. I rejected vCache [9]: it learns a threshold per prompt and puts eviction explicitly out of scope, and because it subsumes calibrated per-cluster thresholds as a lossy special case, FreCoS has no threshold component.

2 Baseline

This section covers why I picked this library, what it ships, and which of its defaults my extension changes. `docs/baseline-justification.md` carries the same material in standalone form, and `docs/baseline-source-map.md` maps each claim to the source line behind it.

2.1 Why GPTCache

I extended GPTCache [10] rather than a more actively maintained semantic cache, for four reasons. First, its most recent release (v0.1.44, August 2024) is frozen and the maintainers have stopped adding integrations, which makes it a better scientific control than a moving target. Second, it fits architecturally, which I verified by reading the vendored source: every component I touch sits behind a named interface, so my extension only adds code (a metadata field via composition, a serve-path gate in one shared decision helper). Eviction is the one exception, reached by monkeypatching `EvictionBase.get()` because that factory is a hardcoded if/elif chain with no lookup table a caller could add to. Appendix A verifies that path against GPTCache's own eviction factory, and I never edit vendored code. Third, GPTCache is the baseline the closest prior art already uses, though in practice I substituted a documented proxy for their policy (Section 5.2). Finally, its default embedding backend runs CPU-deterministically, keeping every run reproducible without a GPU.

2.2 What GPTCache does today

GPTCache serves a cached response when a query is semantically close enough to one already seen, using a single global cosine threshold (0.8 by default). Its eviction policies are purely count-based (LRU, LFU, FIFO, random); none reads generation time or regeneration cost, and none retains metadata on an evicted entry. Creation and last-access timestamps *are* persisted with no downstream consumer, the gap my extension repurposes. Its reporting layer has no admission control and no cost, false-hit, or staleness tracking.

3 Extension design

3.1 One learned artifact, two consumers

FreCoS fits a per-cluster staleness model and uses it at both ends of the cache's lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. Stale-hit-rate, the fraction of served hits whose content was already invalid, is the metric that makes both uses visible.

Clustering. A fixed, seeded k-means runs once per trace over cached-query embeddings from GPTCache's default ONNX model, fit on the calibration split. Every row then takes its nearest centroid. Nothing downstream ever reads the generator's true label. It is recorded only to score the learned assignment by adjusted Rand index (`cluster_ari`, reported with every bracket-style result). Both consumers read the same, possibly wrong, cluster identity off one `EntryMeta` and neither recomputes it, so they cannot disagree.

Fitting. For each cluster, a decay rate λ_c is fit from observed staleness in a held-out calibration split, assuming exponential validity decay: $\Pr[\text{still valid after } \Delta t] = e^{-\lambda_c \Delta t}$. A TTL follows at confidence level c as $\text{ttl} = -\ln(c)/\lambda_c$. Three modes produce this table with the same output shape: *learned* fits each cluster independently by maximum likelihood, falling back to a pooled rate below thirty observations; *global* pools every cluster regardless of sample size; *oracle* takes the rate from the generator's ground truth, an upper bound never available in a real deployment.

All three modes measure each calibration row's validity interval from the generator's `valid_until` field, so on this axis learned and global are no more deployment-realistic than oracle. The fitter is the only reader of that field on any decision path under `gptcache_ext/`: `tests/invariants.py` allowlists it beside the contract that declares the field and the metadata adapter that carries it for after-the-fact scoring, and fails on any other reader. Fitting the table in production needs another source for those intervals: a staleness detector, or a feedback channel that labels a served answer stale (a user correction, a downstream verifier, a revalidation probe against the upstream source).

Consumer 1: the validity gate. On a candidate hit, if the entry's age exceeds its cluster's TTL, it is treated as a miss and the event counts as a stale hit *prevented* rather than served. Age is measured from creation time, never last-access time: staleness is a property of when an answer was generated, not of how recently it was asked for, so a popular but outdated entry is never treated as fresh.

Consumer 2: the eviction value function.

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen\_cost}) \cdot e^{-\lambda_c \cdot \text{age}} \tag{1}$$

Here age is again measured from creation time, and $\kappa = 1000$ log-compresses regeneration cost so its heavy tail cannot dominate the product. The victim is the lowest-value entry, with ties broken by oldest creation time and then by lowest entry id. Unlike GDSF-style formulas and Cortex's LCFU, nothing here divides by `size_bytes`, because eviction runs under an entry-count budget: freeing a 10-byte entry and a 10,000-byte entry both free exactly one slot, so a size term would have nothing to act on. An isolation run under real eviction pressure (a 330-entry cache on a 12,000-query trace, 10 seeds) confirms this: with and without the term, FreCoS is statistically indistinguishable on stale-hit-rate, hit rate, cost and latency alike (`results/ablation/size_term_isolation/`). That run predates the current metric schema and no committed runner regenerates it, as its `summary.md` records.

The frequency term is $\log(2 + \text{freq})$ rather than the more natural $\log(1 + \text{freq})$, and the difference matters more than it looks: $\log(1 + \text{freq})$ is exactly zero for a never-accessed entry, which zeroes the whole product regardless of cost or freshness and makes a freshly written entry the next victim every time. The property test `test_cold_start_entry_scores_above_zero` checks this against the literal formula. $\log(2 + \text{freq})$ has the same concave shape but stays strictly positive at zero frequency.

Because the gate already refreshes anything past its TTL, the decay term does no staleness prevention on its own: it acts as a soft prior among candidates the gate has judged fresh. Section 5.3, the only experiment here that runs the function against competing policies, tests it with the gate off. Table 1 collects every tunable of both consumers, its default, and the section that measures it.

Table 1: Every tunable parameter, its default, and where its effect is measured. `Config` ships $c = 0.95$; the bracketing and ablation experiments run at $c = 0.90$, called out wherever those results appear. Cache size is 412 entries in the bracketing and ablation experiments and 495 in the sweeps; the cache-size sweep identifies 248 entries as its knee (Section 5.4). The similarity threshold, κ, and the thirty-observation fallback are fixed at plausible values and never swept. No experiment here runs at the 1000-entry `Config` default itself.

Parameter	Default	Swept	Measured in
lambda source (mode)	learned	global, learned, oracle	5.1
TTL confidence c	0.95	0.80, 0.90, 0.95, 0.99	5.4
cluster count K	10	5, 10, 20, 50	5.4
cache size (entries)	1000	5%–80% of working set	5.4
similarity threshold	0.8	not swept	n/a
κ (cost log-compression)	1000	not swept	n/a
calibration fallback threshold	30 obs.	not swept	n/a

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator built for this project, and the only workload here. A second one derived from Wikipedia revision history does not clear a feasibility bar fixed in advance: its automatic sentence-change rule agrees with hand labels only 40% of the time, against an 80% go/no-go threshold (`docs/w2-feasibility.md`).

W1 generates queries across several streams inside each topic cluster: canonical queries establishing a ground-truth answer, near-duplicate paraphrases, novel long-tail queries that never repeat, and time-shifted repeats. Every cluster draws its own half-life and regeneration-cost distribution, and at least one repeat per answer lands past its true expiry, giving the gate something to reject. Each trace splits by arrival time, 30% calibration and 70% evaluation, so no parameter is fit on data it is later scored against.

One trace interleaves two profiles rather than running them as separate suites. Paraphrases and time-shifted repeats are the repetitive prompts that stress the hit/miss ratio. The never-repeating long tail can only miss, so it supplies the decisions where the extension's overhead is paid with no hit to offset it. Those queries are novel but short: every W1 query is a single templated question, so prompt length varies only inside a narrow band and nothing here tests how overhead scales with prompt size. Overhead is a single mean per decision over the whole trace (Table 5), not broken out by stream.

One workload-design requirement is easy to get wrong and silently invalidates everything downstream: *cluster identity has to be recoverable from query text alone*, or per-cluster fitting is never exercised. A

template varying only in embedded integers is not: a general-purpose embedder scores such text at 0.6–0.9 cosine similarity regardless of true cluster, putting adjusted Rand index near its chance value of zero (0.02–0.06). W1's canonical text therefore crosses a distinct topic phrase per cluster (51 real-world subjects, e.g. "the Roman Empire", "quantum computing") with a distinct aspect and qualifier per answer (750 combinations). That lifts median `cluster_ari` to 0.42, substantial but imperfect. `tests/test_w1.py` guards it by bounding the seed-0 index to 0.2–0.9: above the 0.02–0.06 a template no embedder separates would give, and below near-perfect separation.

The default generator draws half-life from an exponential distribution, matching the fitter's own assumption exactly. Section 5.1.1 reruns the bracketing experiment with that assumption deliberately violated.

The generator's two ground-truth distributions, regeneration cost and content half-life, are parameterized from commonly cited public figures, not fit to an external source: neither the Azure LLM Inference Trace nor observed update-frequency in a public corpus is reachable from my development environment. The parameterization is therefore a plausibility check, not a real external fit, and a limit on external validity.

4.2 Harness and simulation model

The harness is a pure trace replayer; no language model is ever called. On a miss, response content and regeneration cost are whatever the trace recorded. Miss latency is not. It is drawn from a seeded log-normal scaled by entry size, deterministic per query, a documented placeholder, not a fit to a real trace. Cost and latency are therefore synthetic in different senses, and only latency has a random number generator behind it. Hit latency and extension overhead (index lookup plus gate check) are measured for real, about 1 ms per decision (Table 5), dominated by the cosine search rather than the gate. Reported latency is a hybrid of measured and simulated cost, which keeps runs free and deterministic but is a real limitation.

Lookup uses `benchmarks.semantic_index.SemanticIndex`, brute-force cosine similarity at GPTCache's default 0.8 threshold. The threshold lives in `benchmarks/embedding_pipeline.py` as a module constant, not a `Config` field, so it never reaches the CSV schema and is never swept. A real index rather than exact-match lookup is what makes false-hit-rate, the share of served hits answering a different question, a live metric, and it is high: median 0.89–0.92 across every bracket-style experiment, meaning Section 5.1 and its three follow-ups. The embeddings show why: two aspects of the *same* topic (two distinct questions about Renaissance art, say) score 0.7–0.8 cosine similarity, close enough to cross the match threshold, because to a general-purpose embedder they are related text. That is a recognizable failure mode for a real semantic cache, not a workload artifact. The cache starts empty in every run, and the first 10% of the evaluation split warms it and is excluded from every metric.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry's true expiry. False-hit-rate is the fraction where the served answer's identity does not match the query's. Useful-hit-rate is the fraction of served hits (not of all scored queries) that are both correct and non-stale. It comes from a per-row predicate (`benchmarks.metrics.is_useful_hit`), not from $(n_{\text{hits}} - n_{\text{stale}} - n_{\text{false}})/n_{\text{hits}}$. One hit can be both stale and false, so that subtraction double-counts the overlap and goes negative once both rates are large.

Cost saved sums regeneration cost over useful hits only: a hit returning an expired answer, or an answer to a different question, saved no backend call the caller would have accepted. Given how high false-hit-rate runs here, that exclusion keeps `cost_saved_usd` a measurement rather than an upper bound. Adjusted Rand index (`cluster_ari`) measures how well the learned assignment matches the true labels: 1.0 is perfect, 0.0 random.

Two primary comparisons are designated up front, exempt from correction: learned versus global (does per-cluster learning beat pooling), and gate-on versus the LFU floor (does the gate do measurable work). Both hold. Every other Mann-Whitney test is secondary and corrected via Holm-Bonferroni across the ten-member secondary family (`analysis/multiple_comparisons.py`); Appendix C reports raw and adjusted p side by side for all twelve comparisons. No secondary comparison survives correction at $\alpha = 0.05$; the smallest adjusted p in the family is 0.090. Every secondary result below is therefore exploratory, reported for its effect size and its direction rather than as evidence, and the two primaries carry this report's claims.

Unless noted, every configuration ran five independent seeds, each with its own freshly generated 3,000-query trace; the exceptions are the 1,800-query calibration bracket and the 10-seed, 12,000-query size-term isolation run. Results are medians with percentile-bootstrap intervals over the per-seed values (10,000 resamples, fixed seed), and the raw hit and stale-hit counts behind every rate are in the results CSVs. Every table here reports $n = 5$, and at that size the interval is exactly the observed range of the five values. Its exact coverage is 93.75% rather than 95%, so every interval and every error bar labeled 95% CI here is really a range, and $n = 6$ would be enough to remove that.

Group comparisons use Mann-Whitney U with rank-biserial correlation, $r = 2U_a/(n_1 n_2) - 1$, as effect size (`analysis/stats.py`, hand-rolled and unit-tested against known closed-form cases: complete separation gives $r = \pm 1$, identical samples give $r = 0$). Every p-value here is a two-sided normal approximation, not an exact permutation value; at $n_1 = n_2 = 5$ the smallest this implementation returns is 0.00902, which cannot alone distinguish a large effect from a small one, so effect sizes accompany every p-value. A non-significant result is an absence of detected difference, never equivalence; where I claim two conditions behave alike, raw counts back it up rather than a large p-value.

Every experiment ran on the machine recorded in its own `env.json`: an Apple M2 Pro (10 cores), 16 GB RAM, macOS, Python 3.13, a shared development machine rather than a dedicated benchmarking host, so peak RSS is a sampled maximum from a contended process. Read it as an order of magnitude, not a precise footprint.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling?

This experiment tests the core claim behind fitting a rate per cluster at all: does it beat a rate pooled across every cluster, and how far is it from the true rate a real system could never observe? I ran three lambda sources for 5 seeds each on a 3,000-query W1 configuration, gate enabled at $c = 0.90$, FreCoS eviction throughout. Cluster identity comes from real k-means over query embeddings; the oracle arm alone reads the true label internally, since its lambda table is keyed by true cluster identity and has no other meaning once clustering is imperfect.

Table 2: Learned tracks between global and oracle, which is the per-cluster fitting claim this experiment was designed to test. The primary comparison (learned vs. global) is significant with a large effect; the secondary comparison against oracle points the expected way (learned worse than the unreachable ceiling) but does not survive correction (Appendix C). All values are medians over 5 seeds; cost saved is in USD. Median n_hits out of 1890 scored queries is 678/635/661 for global/learned/oracle; see `results/brackets/results.csv` for per-seed values.

Lambda source	Stale-hit-rate	95% CI	Cost saved	cluster_ari	False-hit-rate
Global	0.091	(0.085, 0.155)	0.188	0.419	0.898
Learned	0.068	(0.051, 0.087)	0.191	0.419	0.911
Oracle	0.048	(0.042, 0.053)	0.187	0.419	0.914

In Table 2, learned and global, the primary comparison, separate significantly with a large effect in the expected direction ($U_a = 2.0$, $p \approx 0.028$, $r \approx -0.84$): fitting each cluster's own rate beats pooling every cluster's observations into one. Learned and oracle, a secondary comparison, separate with a large effect in the same raw direction ($U_a = 24.0$, raw $p \approx 0.016$, adjusted $p \approx 0.114$, $r \approx 0.92$): per-cluster fitting does not close the gap to a rate keyed to true cluster identity.

The primary comparison has a built-in limit. W1 draws half-life from the same exponential family the fitter estimates by maximum likelihood, and both are scored against the field the generator planted. So this measures parameter recovery under a correctly specified model, not deployment performance. It does show that per-cluster rates are recoverable from imperfect real cluster assignments, and that recovering them beats pooling. Section 5.1.1 relaxes the family assumption; the scoring field stays.

False-hit-rate stays high in all three arms (median 0.90–0.91), for the same-topic/different-aspect reason of Section 4.2; it caveats every absolute number here, but applies equally to all three arms and so does not confound the comparison between them. Three follow-ups (a scarcer-calibration bracket at 1,800 queries and a 248-entry cache, plotted beside the main scale in Figure 1, and the two misspecification checks below) reproduce the same qualitative pattern with `cluster_ari` in the same 0.395–0.419 range, ruling out calibration sample size and half-life-distribution shape as confounds for the clustering-quality finding.

5.1.1 Misspecification: when the exponential assumption is wrong

Two checks break the fitter's exponential assumption on purpose. The first drew W1's true half-life from a Weibull instead (same mean, shape 2); the second and stronger from a 50/50 mixture of two exponentials per cluster, biased against the fitter's single-rate maximum-likelihood estimate.

Table 3: Both checks reproduce the well-specified bracket's ordering at smaller effect sizes. All four comparisons are secondary: none clears $\alpha = 0.05$ after correction, and learned versus global does not clear it even raw. Weibull: learned vs. global $U_a = 6.0$, raw $p \approx 0.175$, adj. $p \approx 0.352$, $r \approx -0.52$; learned vs. oracle $U_a = 23.0$, raw $p \approx 0.028$, adj. $p \approx 0.170$, $r \approx 0.84$. Mixture: learned vs. global $U_a = 4.0$, raw $p \approx 0.076$, adj. $p \approx 0.303$, $r \approx -0.68$; learned vs. oracle $U_a = 23.0$, raw $p \approx 0.028$, adj. $p \approx 0.170$, $r \approx 0.84$.

Check	Lambda source	Median stale-hit-rate	Median cost saved (USD)	cluster_ari
Weibull	Global	0.033	0.173	0.419
Weibull	Learned	0.024	0.177	0.419
Weibull	Oracle	0.003	0.173	0.419
Mixture	Global	0.189	0.232	0.395
Mixture	Learned	0.154	0.237	0.395
Mixture	Oracle	0.119	0.244	0.395

In Table 3 the adversarial mixture shows the highest absolute stale-hit-rate at every lambda source, as it should: the fitter's single-rate assumption really is wrong for a two-rate mixture, and it costs something. The relative pattern survives both misspecifications, which makes clustering quality and half-life misspecification separate sources of error. `cluster_ari` is essentially unaffected by how half-life is drawn (0.395–0.419 across all three runs), because cluster identity is fixed entirely by the query-text embedding step, upstream of and independent from the half-life model.

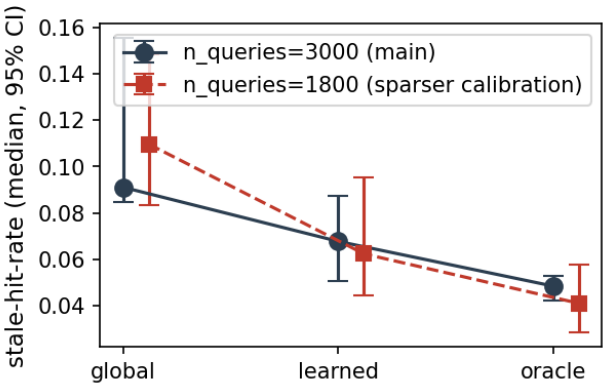

Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the main 3,000-query scale and a roughly 1,800-query sparser-calibration scale. Learned sits between global and oracle at both scales.

5.2 Ablation: isolating the gate from the eviction term

I ran five configurations for 5 seeds each at $c = 0.90$: three gate-disabled eviction policies (LRU, LFU, and a documented substitute for Biton and Friedman's released policy), then the validity gate combined with plain LFU and with full FreCoS eviction.

Table 4: Median stale-hit-rate, hit rate, and false-hit-rate by row, 5 seeds each. Row 2 is the floor. Rows 1–3 agree on every cache metric seed by seed, and so do rows 4–5; both ties follow from eviction never firing at this budget, explained below.

Row	Configuration	Stale-hit-rate	Hit rate	False-hit-rate
1	LRU, gate off	0.546	0.894	0.922
2	LFU, gate off (floor)	0.546	0.894	0.922
3	B&F substitute, gate off	0.546	0.894	0.922
4	LFU, gate on	0.068	0.336	0.911
5	FreCoS eviction, gate on	0.068	0.336	0.911

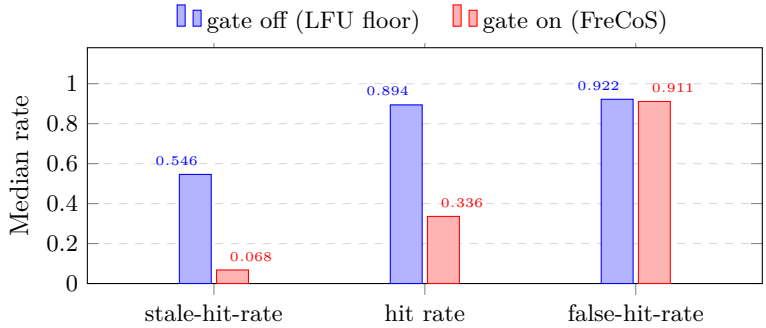

Figure 2: The trade this report is about, in one experiment. Turning the gate on collapses stale-hit-rate and costs most of the hit rate, while false-hit-rate barely moves: the gate is not what makes the absolute numbers poor. Medians over 5 seeds, rows 2 and 5 of Table 4; drawn from `results/ablation/results.csv`.

The gate's own effect, measured by the second primary comparison of row 4 against the floor, is large and significant: with row 4 as the first sample, $U_a = 0.0$, $p \approx 0.0090$, $r \approx -1.00$, perfect separation below the floor, cutting stale-hit-rate 8.07-fold ($\Delta = -0.479$, plotted per row in Figure 3). Most of that needs no per-cluster learning. All three fold-changes below divide a Table 2 arm by this experiment's floor, so they are cross-experiment ratios. Gating on one pooled rate, applied as a single cache-wide TTL, accounts for 6.00x of the reduction by itself (global arm); fitting a rate per cluster adds a further 1.34x, and the oracle rate would add 1.40x on top of that, for 11.3x below the floor overall. Per scored query rather than per served hit, useful hits rise from 0.0164 to 0.0265, or +61.3% (median 31 to 50 useful hits of 1,890 scored queries). The same count taken per served hit reads +343%, because the gate shrinks that denominator. Stated as the relative changes against the floor: stale-hit-rate −87.6%, useful hits per scored query +61.3%, hit rate −62.4%, false-hit-rate −1.2% (Table 4); Table 5 carries the same column for latency, throughput, cost and resource use.

Both ties are trivial, and they have the same cause: eviction never fires in any of the five rows. The resident pool peaks at 256–291 entries against a 412-entry budget, so `select_victim` is called zero times, in every row and at every seed. The gate does not change that, because the harness refreshes a rejected stale entry in place instead of inserting a second copy under a new key: the 987–1138 staleness preventions per gated run rewrite entries that already exist and add none, so the gate cannot manufacture pressure of its own.

Rows 1–3 therefore tie because no policy is ever called to select a victim, not because they agree on one, and rows 4–5 tie for the same reason. What this ablation isolates is the gate alone, not the gate plus an eviction policy. Equation 1 is exercised against competing policies in exactly one experiment here, Section 5.3, where a 25-entry budget holds the cache full throughout and `select_victim` runs 1,385–1,586 times per run across the three arms, and it loses there. The gate-on combination the design intends does run under eviction pressure, at the 82- and 248-entry points of the cache-size sweep, but with no competing policy beside it.

Table 5: Latency and resource use at the LFU floor (row 2) and at gate-on FreCoS eviction (row 5), medians over 5 seeds, with the relative change between them. Latency and throughput are dominated by the simulated miss-latency distribution (Section 4.2). The two cost rows are not: both sum the regeneration cost each trace row records, so spend is a real cost of gating, not a simulation artifact. Overhead and CPU are measured. Extension overhead and cost saved are the only rows that improve, and the text below shows why the overhead and RSS changes are run-to-run variation rather than effects of the gate.

Metric	Row 2 (floor)	Row 5 (gate on)	Change
Extension overhead per decision (ms), measured	1.04	0.82	−20.9%
Latency, mean (ms), simulation-dominated	15.64	95.55	+511%
Latency, p50 (ms), simulation-dominated	0.89	81.42	+9035%
Latency, p95 (ms), simulation-dominated	124.40	280.50	+125%
Latency, p99 (ms), simulation-dominated	267.24	419.42	+56.9%
Throughput (queries/s), simulation-dominated	63.96	10.47	−83.6%
Cost spent (USD), trace-recorded	0.669	4.317	+545%
Cost saved (USD), trace-recorded	0.126	0.191	+51.8%
Peak RSS (MB), sampled	179.3	183.7	+2.5%
CPU (%), measured	61.3	72.8	+18.8%

Peak RSS and extension overhead both have overlapping per-seed ranges (178–183 against 175–

194 MB; 0.79–1.39 against 0.71–1.12 ms), so neither difference in the table is an effect of the gate. The latency and throughput regressions are large because the gate turns many hits into misses and a simulated miss costs far more than a real cache-side decision. The p50 moves furthest and shows that plainest: the median request is a sub-millisecond hit at the floor and a simulated miss with the gate on. None of these says anything about real end-to-end latency, which is also why I plot no latency distribution: its shape would be the miss-cost random number generator's, not the cache's. The p50, p95 and p99 rows above are as far as the measured part of this pipeline can honestly be pushed.

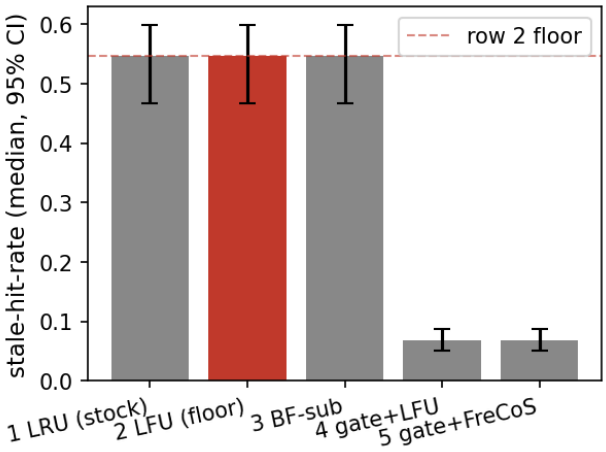

Figure 3: Stale-hit-rate across all five ablation rows, with 95% confidence intervals. Row 2, LFU with the gate disabled, is the floor of comparison.

5.3 A fair test of cost-aware eviction

This experiment gives FreCoS's cost and decay terms the eviction pressure no ablation row above supplies. The gate is off and three eviction policies run for 5 seeds each at a 25-entry cache, about 1.5% of the 1,650 distinct answers this trace scale produces, small enough that the cache reliably fills. It does: the cache sits at 25 of 25 entries throughout, 1,325–1,515 misses occur per run out of 1890 scored queries, and `select_victim` runs 1,385–1,586 times per run for each policy. That is real eviction pressure, and the only place where Equation 1 runs against a competing policy.

Table 6: With real eviction pressure, FreCoS's cost term is indistinguishable from the LFU floor on cost saved, and LRU beats both on cost saved and on stale-hit-rate. Cost saved counts useful hits only (Section 4.3), which is why the absolute values are small.

Policy	Median cost saved (USD)	95% CI	Median stale-hit-rate
FreCoS	0.138	(0.084, 0.234)	0.420
LFU	0.120	(0.075, 0.216)	0.526
LRU	0.213	(0.133, 0.365)	0.092

In Table 6, FreCoS versus LFU on cost saved: $U_a = 13.0$, $p \approx 0.917$, $r \approx 0.04$, no effect at all. The two medians sit well inside each other's confidence intervals and the per-seed values interleave. This is the eviction term's best case, and it does not separate from a plain frequency count.

LRU beats FreCoS on cost saved ($U_a = 20.0$, $p \approx 0.117$, $r \approx +0.60$, large effect, not significant at $n = 5$) and is strictly lower on stale-hit-rate ($U_a = 0.0$, $p \approx 0.0090$, $r \approx -1.00$, perfect separation), as it is against the LFU floor on stale-hit-rate ($U_a = 0.0$, $p \approx 0.0090$, $r \approx -1.00$). All three are secondary comparisons and none survives correction (Appendix C), so read them as effect sizes with a consistent direction. Useful-hit-rate follows the same ordering (0.123 for LRU, 0.087 for LFU, 0.077 for FreCoS), and that is what the cost column measures, since cost saved sums regeneration cost over useful hits only.

The mechanism has nothing to do with staleness modeling. Under a 25-entry budget with no gate active, the LRU resident set is close to the 25 most recently touched entries, and since every miss inserts a freshly created entry, a survivor under LRU is usually one created recently. Evicting the least recently used entry is therefore nearly evicting the entry most likely to have expired, and LRU gets that for free, without a staleness model, a calibration split, or a gate.

This is a negative result: FreCoS's cost term buys nothing measurable over LFU here, and both lose to the cheapest policy in the comparison. Biton and Friedman report LRU-class policies underperforming on semantic workloads, which is why LFU is the floor everywhere else here; on this trace the opposite holds, and the reason is the trace, not the value function. Separating the two needs a workload where creation age and access recency come apart, which W1 does not provide.

5.4 Cache size and the correctness-efficiency trade-off

Three sweeps run here: cache size, the gate's TTL confidence, and cluster count. The second produces the correctness-efficiency trade-off this section is named for. I swept cache size across 5 points spanning 5%–80% of the workload's distinct-answer working set, gate on with FreCoS eviction throughout. Hit rate reaches its ceiling at 15% of the working set (248 entries, Figure 4) and flattens completely from there: every point from 248 entries on produces identical per-seed hit counts. The knee is computed from the sweep's own data by `analysis/fig3_cache_size.py`, as the smallest cache size whose median hit rate is within 1% of every larger size's, so figure and caption cannot drift from the CSV. It is the smallest qualifying *sweep point*, not a resolved knee: the next point down is 82 entries and does not qualify, so all the data can support is a knee somewhere above 82 and at most 248.

The knee is a property of this sweep, not the size the rest of the report runs at: the TTL and cluster-count sweeps use 495 entries, the bracketing, misspecification and ablation experiments 412, and the scarcer-calibration bracket 248 on its smaller trace. The first two sit inside the flat region, above the 256–291 entries the pool reaches at this trace scale, which is why neither of them ever evicts. The second sweep varied the gate's TTL confidence (Table 7).

Table 7: Useful-hit-rate (correct and non-stale, among served hits only; Section 4.3) rises monotonically with TTL confidence, moving opposite to false-hit-rate, while stale-hit-rate falls to zero over the same range.

TTL conf.	Med. stale-hit-rate	Med. hit rate	Med. useful-hit-rate	Med. false-hit-rate
0.80	0.142	0.486	0.042	0.946
0.90	0.068	0.336	0.080	0.911
0.95	0.046	0.231	0.139	0.857
0.99	0.000	0.079	0.458	0.542

Useful-hit-rate rises from 0.042 at confidence 0.80 to 0.458 at 0.99 (0.95 vs. 0.99: $U_a = 0.0$, raw $p \approx 0.0090$, adjusted $p \approx 0.090$, $r \approx -1.00$, perfect separation), and false-hit-rate falls from 0.946 to 0.542 over the same range. Both movements happen on the serve path, not in the pool. The gate is an admission filter over a fixed resident set: eviction never fires at the 495-entry budget these sweeps use, with `select_victim` recording zero calls at all four confidence points. What c changes is how many candidate hits clear the age check.

Served hits fall from a median 919 to 149 as preventions rise from 761 to 1,532, and the hits a tighter gate keeps are the young ones. Those are both likelier to be unexpired (stale-hit-rate 0.142 to 0.000) and likelier to carry the answer actually asked for (0.946 to 0.542), since a recently written entry is usually a near-duplicate of the query that wrote it: the same recency structure Section 5.3 traces. Correctness on both axes therefore comes straight out of hit rate, which falls from 0.486 to 0.079. That trade-off, plotted in Figure 5, and not either axis alone, is the practical result of this sweep.

The third sweep varied cluster count at 5, 10, 20, and 50. Hit rate and stale-hit-rate stay non-monotone with heavily overlapping CIs, and `cluster_ari` is non-monotone too (0.451 at $K = 5$, 0.272 at $K = 20$, 0.320 at $K = 50$). This is not topic-vocabulary exhaustion: the generator's 51 topics comfortably cover every K tested. The likelier explanation is fewer calibration observations per cluster at a fixed trace size as ground-truth clusters get smaller, but at 5 seeds per point that is a wide-CI reading, not a confirmed effect. It therefore sits in a supplementary table (`analysis/figures/supplementary.csv` for the rates, `results/sweeps/cluster_k/results.csv` for `cluster_ari`) rather than a headline figure, clearing neither this report's effect-size nor its confidence-interval bar.

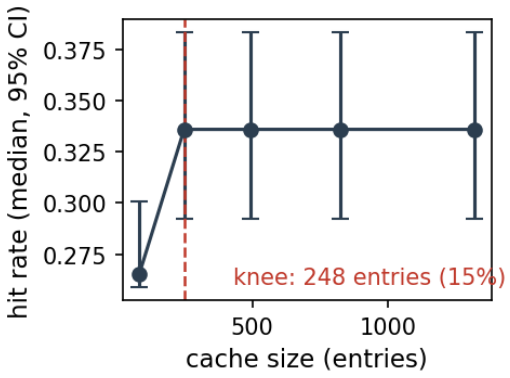

Figure 4: Hit rate against cache size, with the identified knee at 15% of the working set (248 entries) marked.

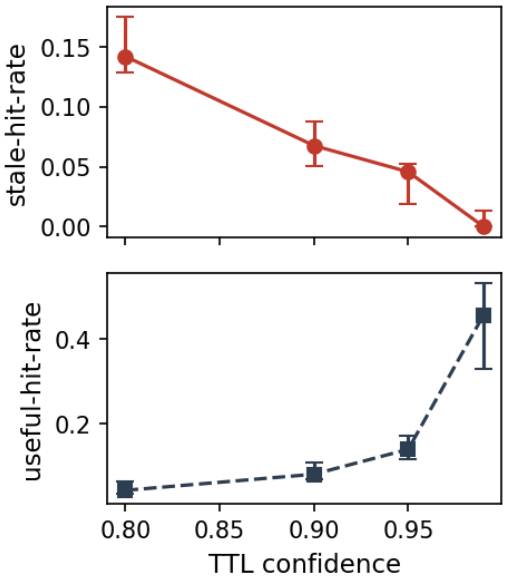

Figure 5: Stale-hit-rate and useful-hit-rate against the gate's target confidence. Stale-hit-rate falls to zero and useful-hit-rate rises from 0.04 to 0.46.

6 Discussion

Table 8: The questions this report set out to answer, and where each is answered. Full statistics stay in the sections named.

Question	Result	Verdict	Where
Does freshness gating cut stale hits?	0.546 to 0.068, 8.07-fold, perfect separation across seeds	Yes	5.2
Does a rate per cluster beat one pooled rate?	0.091 to 0.068, $r \approx -0.84$, $p \approx 0.028$	Yes	5.1
Does cost-aware eviction beat LFU?	0.138 against 0.120 cost saved, $r \approx 0.04$; LRU beats both	No	5.3
Does FreCoS improve conventional cache performance?	Hit rate 0.894 to 0.336, spend up 6.45x	No, that is the trade	5.2
Are the absolute rates production-realistic?	False-hit-rate 0.89–0.92 throughout	No, synthetic workload	4.2

Table 8 collects the answers. The three findings behind them differ sharply in strength. The gate is the strongest. It cuts stale-hit-rate roughly 8-fold against the LFU floor with perfect separation across seeds, and does so *independently of clustering quality*, because the same assignment is used at calibration and at serve time: a wrong but consistent cluster still yields a usable TTL. That is a structural argument, not a measured one. No experiment here varies clustering quality against a gate-off control.

Per-cluster learning is the more interesting finding. Learned rates beat a pooled rate significantly, the first of the two primary comparisons, and fall short of oracle by a margin large in effect size but exploratory once corrected, which is what a working fitter should produce given an imperfect signal (median `cluster_ari` 0.42). It also depends on a workload property that is easy to get wrong: without cluster identity recoverable from query text, per-cluster fitting is never exercised and a suite can report a clean-looking null result that measures nothing (Section 4.1). Future work on a real corpus should verify `cluster_ari` before trusting any per-cluster result.

Cost-aware eviction is the weakest of the three, and it does not work. In the setting built to give it every advantage, FreCoS's cost term is indistinguishable from the pure-frequency floor on cost saved, and plain LRU beats both, because access recency tracks creation age on this trace. With that structure, a policy with no staleness model at all is hard to beat.

Sensitivity to parameters splits the same way. Across the 13 points of the three sweeps in Section 5.4, gate-on stale-hit-rate never exceeds 0.142, at the loosest TTL confidence tested; the other twelve sit between 0.000 and 0.089, far below the 0.546 the ungated floor reaches in Section 5.2. That is a cross-experiment comparison at a different budget, and no sweep runs a gate-off arm, so read it as the gate held on across parameter space rather than re-tested at every point. Only TTL confidence moves an outcome materially, trading stale-hit-rate against useful-hit-rate monotonically, which makes it a deployment knob rather than a tuned constant. Three tunables stay at their defaults, marked so in Table 1: the similarity threshold, κ, and the calibration fallback count. Nothing here speaks to their effect.

Two limitations cut across everything above. First, the absolute usefulness of this cache is poor. False-hit-rate sits at 0.89–0.92 in every bracket-style experiment (Section 4.2) and moves from 0.946 to 0.542 across the TTL sweep of Section 5.4. Every claim in this report is therefore restricted to relative comparisons within an experiment, where the rate applies across arms alike, and no absolute number here should be read as production-realistic. The damage is bounded, though. The gate is not the cause: the ungated floor runs at 0.922 against the gated 0.911, so the rate belongs to the index and the workload. And the knob most likely to move it, the 0.8 similarity threshold, is fixed and never swept, so this is an untested axis, not a measured ceiling. Second, circularity: stale-hit-rate is scored against `valid_until`, and that same field supplies the oracle arm's lambda *and* the calibration intervals the learned and global arms fit, so the metric, its ground truth, the oracle ceiling, and the first primary comparison's fit all rest on one generator I wrote myself. Relative comparisons within an experiment stay valid, since both sides are scored on the same ground truth, but no number here is a claim about staleness detection under an independent notion of correctness, and there is no external ground truth to check one: the Wikipedia alternative measures 40% rule agreement against an 80% bar (Section 4.1). Neither prior-art comparator, Biton and Friedman's released policy nor Cortex's LCFU, ran on this workload, so the eviction axis is measured only against count-based policies and FreCoS itself.

Finally, scale: 3,000 queries and 5 seeds per configuration, chosen because embedding dominates runtime. The cost is statistical power: several comparisons show a large effect and a consistent direction without clearing $p < 0.05$ at $n = 5$ (LRU vs. FreCoS on cost saved, most notably), and each is reported with its effect size beside the non-significant p-value rather than as a null result.

7 Conclusion and future work

Semantic caches are not evaluated on whether a served hit is still true. This report names that gap stale-hit-rate, and FreCoS addresses it with two components, one for each gap in Section 1.1. The validity gate does the job: an 8-fold cut in stale-hit-rate against the LFU floor, with perfect separation across seeds. The per-cluster learning that sharpens it holds up too, beating pooling with a large, significant effect and trailing oracle by an exploratory margin, though as parameter recovery inside the generator's own family rather than as a deployment result. The eviction term does not: where its value function actually runs, its cost-awareness is indistinguishable from a pure-frequency floor, and plain LRU beats both on this trace for reasons unrelated to staleness modeling. The gate is expensive on every conventional metric, and Table 5 prices it.

The most important next step is the false-hit-rate gap: same-topic, different-aspect queries still land close to the index's match threshold, so a more semantically distant aspect vocabulary, or a real corpus in place of a synthetic template, would likely reduce false-hit-rate without touching the clustering pipeline. The eviction result points somewhere specific too: a workload whose creation-age and access-recency orderings disagree, so the value function has something to supply that recency does not. Beyond that: running Biton and Friedman's policy and Cortex's LCFU against the real index this harness provides; calibrating the generator against a real inference trace; and, mattering most, breaking the circularity between stale-hit-rate and its generator-supplied ground truth. Dynamic re-clustering, a FreshCache-style staleness budget, byte-budgeted eviction (where Equation 1's absent size term would carry meaning), and admission control remain future work.

Appendix A: correctness evidence

Correctness is checked at four levels, 101 tests in all, rerun by CI on every commit. A reference oracle (`tests/oracle/reference_cache.py`) implements the same hit/miss/stale decision by dict-and-linear-scan, sharing only the `contracts` dataclasses and a similarity helper, and `tests/test_oracle_agreement.py` cross-checks every `decide()` branch against it over a 5,000-query

trace under a test-only ground-truth gate. Separately, `tests/test_stock_parity.py` runs 10,000 queries through both `decide()` with the gate disabled and stock GPTCache, and records zero divergences. A callable invariant suite (`tests/invariants.py`) runs inside both the unit tests and `benchmarks/harness.py` on every experimental run, checking budget bounds and argmin eviction selection wherever eviction fires. Two further checks, that no entry is served past its cluster's TTL and that staleness decisions derive from creation time rather than last-access time, run only on gate-on runs: with the gate off there is no gate to check, which is why the ungated rows of Table 4 serve stale hits at all.

Property-based tests (`tests/test_frecos.py`) check that the eviction value function is monotonic in each term independently. Finally, `tests/test_gptcache_integration.py` drives a real `gptcache.manager.data_manager.SSDataManager` through enough writes to force eviction and confirms the entry GPTCache's own factory evicts is the one `FreCoSEviction.select_victim()` names. The same selector sits on both sides of that assertion, so it establishes reachability through GPTCache's real eviction path rather than agreement between two implementations.

Appendix B: code and data artifacts

This repository is public at https://github.com/LiorLotan2/frecos. A `Dockerfile` and `environment.yml` (or `requirements.txt`) build a working environment from a clean clone, and `.github/workflows/ci.yml` reruns the full test suite, a benchmark smoke test, an experiment-path smoke test, and a figure-consistency check on every commit; the half-hour performance suite itself runs on demand, not per commit. `benchmarks/samples/` holds committed sample logs, reproducible with `make bench-smoke`; `make experiments` reruns all seven experiments behind Section 5 in dependency order, and `make figures` regenerates every figure from the resulting CSVs, none hand-edited. Table 9 lists every code, data and supporting-note artifact this report cites.

Table 9: Where every artifact referenced in this report lives in the repository.

Artifact	Location
Serve-path decision seam	`gptcache_ext/pipeline.py`
Additive metadata storage	`gptcache_ext/metadata.py`
Staleness model (fitter, gate)	`gptcache_ext/staleness/`
Real clustering (k-means, ARI)	`gptcache_ext/staleness/embedder.py`, `clusters.py`, `assign_real_clusters.py`, `cluster_accuracy.py`
Eviction (FreCoS and baselines)	`gptcache_ext/eviction/`
Semantic index (cosine, 0.8)	`benchmarks/semantic_index.py`
Workload generator (W1)	`workloads/w1_synthetic/`
Harness, metrics, runners, sample logs	`benchmarks/harness.py`, `metrics.py`, `runners/`, `benchmarks/samples/`
Raw results, summary, environment	`results/*/results.csv`, `summary.md`, `env.json`, `results/ablation/size_term_isolation/`, `analysis/figures/supplementary.csv`
Statistics, correction, figure scripts	`analysis/stats.py`, `multiple_comparisons.py`, `analysis/fig*.py`, `make_supplementary.py`
Oracle, agreement, parity, invariants	`tests/oracle/reference_cache.py`, `tests/test_oracle_agreement.py`, `tests/test_stock_parity.py`, `tests/invariants.py`, `tests/test_frecos.py`, `tests/test_w1.py`
GPTCache eviction-factory integration	`tests/test_gptcache_integration.py`
Baseline, source map, related work	`docs/baseline-justification.md`, `baseline-source-map.md`, `related-work.md`
W1 parameterization, W2 feasibility	`docs/w1-calibration.md`, `docs/w2-feasibility.md`
Reproduce experiments / figures / report	`make experiments`, `make figures`, `make report`
Environment and CI	`Dockerfile`, `environment.yml`, `requirements.txt`, `.github/workflows/ci.yml`

Appendix C: multiple-comparison correction

P1 and P2 in Table 10 are the two primaries designated in Section 4.3, exempt from correction, and both hold. The smallest adjusted p among the ten secondaries, 0.090, is tied three ways at the floor a normal-approximation U test returns at $n_1 = n_2 = 5$. U_a and r take the first-named sample as `sample_a`; reversing a pair maps $U_a \to n_1 n_2 - U_a$ and $r \to -r$, and the body quotes whichever orientation its own sentence names first.

Table 10: Every comparison in the report's family, with raw and Holm-adjusted p.

	Comparison (metric)	U_a	r	raw p	adj. p
P1	brackets learned vs. global (stale)	2.0	−0.84	0.0283	exempt
P2	ablation floor row 2 vs. row 4 (stale)	25.0	1.00	0.0090	exempt
S1	brackets learned vs. oracle (stale)	24.0	0.92	0.0163	0.1141
S2	Weibull learned vs. global (stale)	6.0	−0.52	0.1745	0.3516
S3	Weibull learned vs. oracle (stale)	23.0	0.84	0.0283	0.1697
S4	mixture learned vs. global (stale)	4.0	−0.68	0.0758	0.3032
S5	mixture learned vs. oracle (stale)	23.0	0.84	0.0283	0.1697
S6	FreCoS vs. LFU (cost saved)	13.0	0.04	0.9168	0.9168
S7	FreCoS vs. LRU (cost saved)	5.0	−0.60	0.1172	0.3516
S8	FreCoS vs. LRU (stale)	25.0	1.00	0.0090	0.0902
S9	LRU vs. LFU (stale)	0.0	−1.00	0.0090	0.0902
S10	TTL $c = 0.95$ vs. $c = 0.99$ (useful)	0.0	−1.00	0.0090	0.0902

References

[1] D. D. Biton and R. Friedman. *From Exact Hits to Close Enough: Semantic Caching for LLM Embeddings*. arXiv:2603.03301, 2026.

[2] C. Ruan, C. Bi, K. Zheng, Z. Shi, X. Wan, and J. Li. *Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching*. arXiv:2509.17360, 2025.

[3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.

[4] C. Wang, X. Liu, Y. Zhu, A. Youssef, P. Nagpurkar, and H. Chen. *Category-Aware Semantic Caching for Heterogeneous LLM Workloads*. arXiv:2510.26835, 2025.

[5] M. Mansoor, T. Ahmad, and Y.-C. Yoon. *Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs*. arXiv:2607.04281, 2026.

[6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.

[7] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.

[8] G. Einziger, R. Friedman, and B. Manes. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.

[9] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.

[10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. https://github.com/zilliztech/GPTCache, 2024. Version 0.1.44, commit bae7ffeef774e762d9d4e60fce70be00011188a6.